FREE PROMPT PACK

The Security Prompt Pack

Copy-Paste Prompts to Catch What Your AI Misses

Here's the thing about AI coding tools: they build what you ask for, but they don't think about who might try to break it. You say "build me a login page" and you get a login page. You don't get rate limiting, you don't get secure token storage, and you definitely don't get someone asking "what happens if a user changes the ID in the URL?"

That's what this prompt pack is for. Each section describes a specific security gap that AI tools leave wide open, explains why it matters in plain language, and suggests prompts you can try. The prompts aren't just things to copy and paste — they're there to teach you the kinds of questions you should be asking and thinking about as you build. The more you use them, the more you'll start catching these issues on your own.

I made this because I kept seeing the same mistakes over and over in AI-generated code. The same missing checks, the same exposed secrets, the same unprotected endpoints. These prompts are the questions your AI tool should be asking itself but isn't.

How to use this prompt pack

1. Build your feature first — get it working before worrying about security.
2. Open this PDF and find the relevant section for what you just built.
3. Copy the prompt and paste it into your AI tool.
4. Let the AI fix what it finds, then run the prompt again. It catches layers.
5. Repeat until the AI comes back clean. Then move to the next feature.

What's inside

1 **Frontend-Only Validation**
Your AI validates forms in the browser but forgets the backend

2 **Hardcoded Secrets & API Keys**
Your AI puts passwords and keys directly in the code

3 **Authentication & Session Security**
Your AI builds login but skips token expiry, logout, and session safety

4 **Missing Permission Checks**
Your AI builds features without checking who's allowed to use them

5 **Sensitive Error Messages & Data Leaks**
Your AI shows technical details to users when things break

6 **Injection Attacks (SQL, XSS, CSRF)**
Your AI doesn't protect forms and endpoints from malicious input

7 **File Upload Security**
Your AI accepts any file without checking what it actually is

8 Rate Limiting & Brute Force
Your AI builds login and APIs with no limit on failed attempts

9 HTTPS & Transport Security
Your AI sends data in plain text or mixes secure and insecure content

10 Data Privacy & PII Handling
Your AI stores personal data without encryption or access controls

11 Insecure Configuration & Defaults
Your AI uses dev-mode settings that aren't safe for production

12 Outdated & Vulnerable Dependencies
Your AI installs packages with known security holes

13 Logging, Monitoring & Audit Trails
Your AI builds features with no way to detect when something goes wrong

14 The Master Security Review
The one prompt to run after every single feature you build

1 FRONTEND-ONLY VALIDATION

When you ask your AI tool to build a form — a signup form, a contact form, a checkout form — it almost always adds validation on the frontend. That means the form checks things like "is this a valid email?" or "is this field empty?" before the user clicks submit. But here's the problem: the frontend runs in the user's browser, and anything in the browser can be bypassed.

A technical person (or a bot) can skip your frontend entirely and send data directly to your backend. If your backend doesn't also validate that data, it'll accept literally anything: empty fields, fake emails, SQL commands disguised as usernames. The frontend validation is for user experience. The backend validation is for security. You need both.

> **What can go wrong**
>
> A user submits a form with a script injected into the "name" field. The frontend never checked it because they bypassed the browser form. The backend saves it to the database. Later, when another user views that name on a page, the script executes in their browser — stealing their session or redirecting them. This is called Cross-Site Scripting (XSS), and it starts with missing backend validation.

Prompt 1: Check if backend validation exists

COPY THIS PROMPT

```
Review every form and user input in this project. For each one, tell me: (1) Is there frontend
validation? (2) Is there backend validation? (3) What specific checks does each one do? List any
inputs where the backend accepts data without validating it first. Those are the vulnerabilities I
need to fix.
```

Prompt 2: Add backend validation to an existing feature

COPY THIS PROMPT

```
This feature has form validation on the frontend but not on the backend. Add backend validation
for every field: check that required fields are present, data types are correct, strings don't
exceed reasonable length limits, email fields are valid emails, and numeric fields are within
expected ranges. Reject anything that fails validation with a clear error response — don't just
let it through.
```

Prompt 3: Full input sanitization

COPY THIS PROMPT

```
For every place this app accepts user input — forms, search bars, URL parameters, file uploads,
query strings — make sure the input is sanitized before it's stored or displayed. All user input
should be treated as plain text, never as executable code or database commands. Use parameterized
queries for all database operations. Escape any HTML characters before displaying user-generated
content.
```

Prompt 4: Validate data types and ranges

COPY THIS PROMPT

```
Review every API endpoint that accepts data. For each field, verify that the backend checks: (1)
The data type matches what's expected (string, number, boolean, date). (2) Strings have maximum
length limits to prevent abuse. (3) Numbers are within valid ranges. (4) Dates are valid and in
expected format. (5) Enum fields only accept allowed values. (6) Arrays have a maximum length.
Reject anything outside these bounds with a 400 error.
```

Prompt 5: Add rate-aware validation

```
Add validation that detects and blocks abuse patterns: (1) If the same form is submitted more than
10 times in a minute from the same user or IP, temporarily block submissions. (2) If a field
receives data that looks like an attack (SQL keywords, script tags, extremely long strings), log
the attempt and reject it. (3) Add a honeypot field that's hidden from real users but filled by
bots — reject any submission that includes it.
```

COPY THIS PROMPT

Tip: After the AI adds backend validation, test it yourself. Open your browser's DevTools, go to the Network tab, find the form submission request, and try resending it with modified or missing data. If the backend accepts it, the validation isn't working.

2 HARDCODED SECRETS & API KEYS

When your app connects to external services — a payment processor, an email service, an AI API, a database — it uses a secret key to authenticate. AI tools regularly put these keys directly in the source code as plain text. Sometimes in frontend files, where they're visible to literally every person who visits your site.

The correct approach is environment variables: instead of writing the actual key in your code, you use a placeholder label (like process.env.STRIPE_SECRET_KEY), and the real value lives in a separate .env file that never gets committed to Git or pushed to GitHub.

> **What can go wrong**
>
> Bots scan every public GitHub repository looking for accidentally committed API keys. When they find one, they can use it within minutes — generating thousands of dollars in API charges, sending emails from your account, or accessing your database. Even if you delete the key in a later commit, Git remembers every previous version. You have to rotate (regenerate) the key, not just delete it from the code.

Prompt 1: Scan for hardcoded secrets

```
Scan this entire project for any hardcoded API keys, passwords, database URLs, tokens, webhook
secrets, or other sensitive values. Check every file — including config files, test files, utility
files, and comments. For each one you find, tell me: (1) what file it's in, (2) whether it's a
frontend or backend file, (3) how to move it to an environment variable. Anything in a frontend
file is especially critical because it's visible to every visitor.
```

COPY THIS PROMPT

Prompt 2: Set up environment variables correctly

```
Move all secrets in this project to environment variables. Create a .env file with clear,
descriptive variable names (like OPENAI_API_KEY, SUPABASE_URL, STRIPE_SECRET_KEY). Update every
file that references these values to use the environment variable instead. Make sure .env is in
.gitignore. Create a .env.example file with the variable names but no actual values, so other
developers know what's needed.
```

COPY THIS PROMPT

Prompt 3: Check for secrets in frontend code

```
Search every frontend file in this project — anything that runs in the browser (React components,
Next.js pages, client-side scripts, public folders) — for any API keys, tokens, or secrets. These
are visible to every visitor via the browser's developer tools and must be moved to backend API
routes or server-side functions immediately. List every instance you find.
```

COPY THIS PROMPT

Prompt 4: Verify .gitignore and Git history

COPY THIS PROMPT

```
Check if .env is in .gitignore. Then check the full Git commit history to see if .env or any file containing secrets was ever committed — even if it was later removed. Use: git log --all --full-history -- .env and git log --all -p --diff-filter=A -- '*.env'. If any secrets were ever committed, list them so I can rotate those keys immediately.
```

Prompt 5: Separate public vs secret keys

COPY THIS PROMPT

```
Review every API key and token in this project. Some services have two types of keys: public/publishable keys (safe to use in frontend code) and secret keys (must never be in frontend code). For each key in this project, tell me: (1) which service it belongs to, (2) whether it's a public or secret key, (3) whether it's currently in a frontend or backend file. Move any secret keys that are in frontend files to backend API routes immediately.
```

Prompt 6: Handle API keys safely with AI tools

COPY THIS PROMPT

```
I need to add [service name] to this project. Set up the integration using an environment variable for the API key. Store the variable name in .env with a placeholder value. Never put the actual key value in any code file. Show me where in my hosting platform settings I'll need to add the real value for production. Remind me to never paste the actual key into this chat.
```

Tip: Never paste your actual API key values into your AI tool's chat window. The AI only needs the variable name (process.env.MY_API_KEY) to write the code. You add the real value to your .env file yourself.

3 AUTHENTICATION & SESSION SECURITY

Authentication is how your app confirms that a user is who they say they are. Session management is how your app remembers them after they log in. AI tools can build a working login page quickly, but they almost always skip the details that make it secure: token expiration, secure cookie settings, proper logout, password requirements, and protection against session hijacking.

A login system that works is not the same as a login system that's safe. If your tokens never expire, a stolen token gives an attacker permanent access. If your logout doesn't actually invalidate the session, the user thinks they're logged out but their session is still usable. These are the gaps AI tools leave wide open.

What can go wrong

A user logs into your app from a public computer at a library. They click "log out" and walk away. But your AI-built logout only clears the token from the browser — it doesn't invalidate the token on the server. The next person who sits down can find the token in the browser's storage or network history and use it to access the account. The original user has no idea they're compromised.

Prompt 1: Full authentication audit

COPY THIS PROMPT

```
Review the entire authentication system in this app. Check: (1) How are passwords stored — are they hashed with bcrypt or argon2, not stored in plain text? (2) Do tokens have an expiration time? What is it? (3) Are tokens stored securely (httpOnly cookies, not localStorage)? (4) Does the logout function actually invalidate the session/token on the server, or just delete it from the browser? (5) Is there protection against brute force login attempts? (6) Are password reset tokens single-use and time-limited? List every issue you find.
```

Prompt 2: Fix token and session management

```
Review the session/token configuration in this app. Make sure: (1) Access tokens expire after a short period (15-30 minutes). (2) Refresh tokens expire after a reasonable period (7-30 days). (3) Tokens are stored in httpOnly, secure, sameSite cookies — not in localStorage or sessionStorage (those are accessible to any JavaScript on the page). (4) The logout endpoint invalidates the token on the server side, not just the client. (5) When a user changes their password, all existing sessions are invalidated.
```

COPY THIS PROMPT

Prompt 3: Add password security requirements

```
Review the signup and password change forms. Add these requirements: (1) Minimum 8 characters. (2) Check passwords against a list of commonly breached passwords (use the haveibeenpwned API or a local list of the top 10,000 common passwords). (3) Don't enforce arbitrary complexity rules (like requiring a symbol) — length matters more. (4) Rate-limit password attempts to 5 per minute. (5) Never log or display passwords in plain text anywhere — check console.log statements and error messages.
```

COPY THIS PROMPT

Prompt 4: Secure the password reset flow

```
Review the password reset flow end to end. Check: (1) The reset token is random, long, and unpredictable. (2) The token expires after 15-30 minutes. (3) The token can only be used once. (4) After a successful reset, all other sessions for that user are invalidated. (5) The reset email does not reveal whether an account exists — always show 'If an account exists, we sent a reset link' regardless. (6) The reset page validates the token before showing the form.
```

COPY THIS PROMPT

Prompt 5: Protect against session fixation and hijacking

```
Add these session security measures: (1) Generate a new session ID after every successful login (prevents session fixation). (2) Bind sessions to the user's IP or user agent and flag or invalidate sessions that suddenly change. (3) Add a 'log out everywhere' feature that lets users invalidate all active sessions. (4) Set the secure flag on all cookies so they're only sent over HTTPS. (5) Set sameSite=strict or sameSite=lax on authentication cookies to prevent CSRF attacks.
```

COPY THIS PROMPT

Tip: Open your browser's DevTools, go to Application > Cookies, and look at your auth cookies. If they don't have httpOnly, secure, and sameSite flags, they're vulnerable. Also check localStorage — if tokens are stored there, any script on your page can steal them.

4 MISSING PERMISSION CHECKS

This is the #1 most common security vulnerability on the internet — it's called Broken Access Control, and it's been the top item on the OWASP Top 10 for years. Your AI builds features that work for the correct user. But it almost never adds code to block the wrong user. If someone changes the ID in the URL from /orders/123 to /orders/124, they might see someone else's order.

The frontend might hide the button, but hiding a button isn't security — the backend endpoint is still wide open. Anyone with basic technical knowledge can send requests directly to your API, bypassing your entire frontend.

What can go wrong

A user discovers that your app uses sequential IDs in URLs (order/1, order/2, order/3). They write a simple script that loops through every number and downloads every order in your system — names, addresses, purchase history. Your app had a login system, so it checked that they were logged in. But it never checked that each order belonged to them. This is called an Insecure Direct Object Reference (IDOR) and it's embarrassingly common.

Prompt 1: Full permission audit

```
Review every API endpoint and server action in this project. For each one, tell me: (1) Does it check that the user is authenticated (logged in)? (2) Does it check that the user is authorized to access this specific resource (not just any resource of this type)? (3) What happens if an unauthenticated or unauthorized user sends a request directly to this endpoint? Flag any endpoint that's missing either check.
```

Prompt 2: Add ownership checks to a feature

```
This feature lets users view, edit, and delete their [orders/recipes/documents]. Add ownership verification to every endpoint: before returning, updating, or deleting any record, check that the requesting user's ID matches the owner ID on that record. If it doesn't match, return a 403 Forbidden error. Do not just hide buttons on the frontend — enforce this on the backend.
```

Prompt 3: Protect admin-only features

```
This feature should only be accessible to admin users. Add a role check on the backend: before executing any admin action, verify that the requesting user has an admin role in the database. If they don't, return a 403 Forbidden error — even if they somehow navigated to the admin page. Don't rely on hiding the admin link in the navigation. Protect the endpoint itself.
```

Prompt 4: Test for URL tampering

```
Check every route in this app that uses an ID parameter in the URL (like /users/123, /orders/456, /documents/789). For each one, verify that the backend checks whether the logged-in user is allowed to access that specific ID — not just that they're logged in. A user changing /orders/123 to /orders/124 should never see another user's data.
```

Prompt 5: Add a universal permission middleware

```
Create a reusable middleware or helper function that I can apply to any protected route. It should: (1) verify the user is authenticated, (2) accept a function that checks whether the user is authorized for the specific resource being requested, (3) return 401 if not authenticated and 403 if not authorized, (4) log the attempted unauthorized access for monitoring. Show me how to apply it to an existing route as an example.
```

Prompt 6: Use non-guessable IDs

```
Review every database table in this project. Are any of them using sequential integer IDs (1, 2, 3) that appear in URLs or API responses? If so, switch to UUIDs or another non-sequential ID format for any resource that users can access. Sequential IDs make it trivial to enumerate every record in your system. Show me every table that needs to change and update the code accordingly.
```

Tip: The easiest way to test this yourself — log in as User A, then manually change the ID in a URL to something that belongs to User B. If you can see User B's data, you have a broken access control vulnerability.

5 SENSITIVE ERROR MESSAGES & DATA LEAKS

When something goes wrong in your app, the AI's default behavior is to show the full error — stack traces, file paths, database column names, query details. This is useful during development. It's also a roadmap for anyone trying to hack your app. In production, users should see "Something went wrong." The technical details should be logged on the server where only you can see them.

> **What can go wrong**
>
> An attacker triggers an error on purpose (submitting bad data, requesting a page that doesn't exist). Your app returns a detailed error message showing the database schema, the file path, and the exact query that failed. Now the attacker knows your table names, your column names, and exactly how your database is structured. They use this to craft a targeted injection attack.

Prompt 1: Fix error messages for production

COPY THIS PROMPT

```
Review how this app handles errors. When an API endpoint fails or a page encounters an error, what does the user see? Replace any error responses that expose technical details (stack traces, file paths, database info, query details) with generic, user-friendly messages. Log the full technical error details on the server (not to the browser console) so I can still debug issues.
```

Prompt 2: Clean up console.log statements

COPY THIS PROMPT

```
Search this entire project for console.log, console.error, console.warn, and console.debug statements. List every one that outputs sensitive information: user data, tokens, passwords, session details, API responses with personal data, database queries, or full error objects. Remove them or replace them with server-side-only logging.
```

Prompt 3: Add global error handling

COPY THIS PROMPT

```
Set up a global error handler for this app. On the frontend, add an error boundary that catches unhandled errors and shows a friendly fallback page instead of crashing. On the backend, add a catch-all error handler for API routes that returns a generic error message to the client while logging the full details server-side. Make sure no error response ever includes stack traces, file paths, or database information.
```

Prompt 4: Check what failed requests reveal

COPY THIS PROMPT

```
Test what happens when each API endpoint in this app receives bad data — an invalid ID, missing required fields, wrong data types, an expired token, or an unauthorized request. For each one, show me the exact error response the user would see. Flag any response that reveals internal details like table names, column names, file paths, framework versions, or query structures.
```

Prompt 5: Prevent data leaks in API responses

COPY THIS PROMPT

```
Review every API response in this app. Check: (1) Are you sending entire database records to the
frontend, or only the fields the user needs? (2) Are password hashes, internal IDs, or admin flags
being included in user-facing responses? (3) Are other users' email addresses, phone numbers, or
personal details being exposed in list endpoints? For each endpoint, create an explicit allowlist
of fields that should be returned and strip everything else.
```

Tip: Open your app in an incognito window, open DevTools, and try to break things on purpose. Submit empty forms, visit URLs that don't exist, change IDs in the URL. If you see technical details in the console or network responses, an attacker will too.

6 INJECTION ATTACKS (SQL, XSS, CSRF)

Injection happens when someone types malicious code into a form field and your app treats it as a command instead of plain text. There are three main types you need to know: SQL injection (someone injects database commands), Cross-Site Scripting / XSS (someone injects JavaScript that runs in other users' browsers), and CSRF (someone tricks a logged-in user into performing actions they didn't intend).

All three are in the OWASP Top 10, all three are common in AI-generated code, and all three are preventable — if you know to ask.

What can go wrong

SQL injection: An attacker types ' OR 1=1 -- into a login form. Your app builds the query using string concatenation, so the database interprets it as 'give me everything.' The attacker is now logged in as any user. XSS: An attacker posts a comment containing a script tag. When other users view the comment, the script runs in their browser and sends their session token to the attacker. CSRF: An attacker sends a logged-in user a link that triggers a money transfer — the user's browser automatically includes their auth cookie.

Prompt 1: Check for SQL injection vulnerabilities

COPY THIS PROMPT

```
Review every database query in this project. Are any of them built using string concatenation with
user input? For example, something like: 'SELECT * FROM users WHERE id = ' + userId. If so,
rewrite them using parameterized queries or prepared statements. Show me every query you changed
and explain why the original was vulnerable.
```

Prompt 2: Check for XSS vulnerabilities

COPY THIS PROMPT

```
Review every place in this app where user-generated content is displayed on a page — comments,
usernames, profile bios, search results, any text a user typed that appears in the UI. Is it being
properly escaped or sanitized before rendering? If any user-generated content is inserted as raw
HTML (using innerHTML, dangerouslySetInnerHTML, or equivalent), flag it and fix it. User content
should always be treated as text, never as HTML.
```

Prompt 3: Add CSRF protection

```
Does this app have Cross-Site Request Forgery (CSRF) protection? Check: (1) Are state-changing
requests (POST, PUT, DELETE) protected with CSRF tokens? (2) Are authentication cookies set with
sameSite=strict or sameSite=lax? (3) Does the server verify the Origin or Referer header on
sensitive requests? If any of these are missing, add them. Show me exactly what you changed and
where.
```

COPY THIS PROMPT

Prompt 4: Full injection audit

```
Audit this entire application for injection vulnerabilities. Check: (1) All database queries for
SQL injection — are they all using parameterized queries? (2) All rendered content for XSS — is
user input always escaped before display? (3) All URL parameters for injection — are they
validated and sanitized? (4) All file uploads for path traversal — can a user upload a file with a
malicious filename? (5) All API endpoints for command injection — does any endpoint pass user
input to a shell command? List every vulnerability found and fix each one.
```

COPY THIS PROMPT

Prompt 5: Protect against DOM-based XSS

```
Review all client-side JavaScript for DOM-based XSS. Check if any code reads from URL parameters
(window.location, URLSearchParams), document.referrer, or other user-controllable sources and
writes to the DOM using innerHTML, document.write, or similar. Also check for any eval() calls
with user input. These don't involve the server at all — the attack happens entirely in the
browser. Fix every instance you find.
```

COPY THIS PROMPT

Prompt 6: Protect against open redirects

```
Check every redirect in this app (redirects after login, after form submission, in email links).
Can a user manipulate the redirect URL by changing a query parameter like ?redirect=/dashboard to
?redirect=https://evil-site.com? If so, add a whitelist of allowed redirect destinations and
reject any URL that doesn't match. Open redirects are used in phishing attacks to make malicious
links look legitimate.
```

COPY THIS PROMPT

Tip: A quick test for XSS — type <script>alert('test')</script> into a form field in your app and submit it. If an alert box pops up for you or anyone who views that data, your app has an XSS vulnerability.

7 FILE UPLOAD SECURITY

If your app lets users upload files — profile pictures, documents, attachments — your AI tool probably built the feature without any security checks. That means a user could upload a PHP script disguised as an image, an HTML file containing malicious JavaScript, or a 5GB file that crashes your server. File uploads are one of the most commonly exploited features in web applications.

The file extension (.jpg, .pdf) means nothing — a user can rename any file. You need to check the actual file content, limit sizes, and store uploads outside your application's executable directory.

What can go wrong

An attacker uploads a file named "profile.jpg" that is actually a PHP script. Your server stores it in a public folder. When anyone visits the URL of that "image," the server executes the PHP code instead of displaying a picture. The attacker now has a backdoor to your server — they can read your database credentials, modify files, or take over the entire system. This is called a web shell attack and it's devastatingly common.

Prompt 1: Audit all file upload endpoints

```
Review every file upload feature in this app. For each one, tell me: (1) What file types are allowed? (2) Is the file type checked by extension only, or by actually reading the file's content/MIME type? (3) Is there a file size limit? What is it? (4) Where are uploaded files stored — in a public directory, a private directory, or cloud storage? (5) Are filenames sanitized to prevent path traversal attacks? List every upload endpoint and what's missing.
```

COPY THIS PROMPT

Prompt 2: Add proper file validation

```
For every file upload in this app, add these checks: (1) Validate the file's actual content type by reading the file header (magic bytes), not just the extension. (2) Set a maximum file size (5MB for images, 25MB for documents — adjust for your use case). (3) Only allow specific, expected file types (e.g., jpg, png, pdf). (4) Reject any file with a double extension (like file.php.jpg). (5) Strip all metadata from uploaded images using a library like sharp. (6) Generate a new random filename for every upload — never use the original filename.
```

COPY THIS PROMPT

Prompt 3: Secure file storage

```
Review where uploaded files are stored. Make sure: (1) Files are stored outside the web root — they should never be directly accessible via URL without going through your application. (2) Use cloud storage (like S3 or Supabase Storage) with proper access controls rather than local disk. (3) Serve files through a proxy endpoint that checks if the requesting user has permission to view that file. (4) Set the Content-Disposition header to 'attachment' for file downloads so browsers don't try to execute them.
```

COPY THIS PROMPT

Prompt 4: Scan uploads for malicious content

```
Add a check that scans uploaded files for malicious content before storing them. For images: re-encode the image using a library (like sharp for Node.js or Pillow for Python) — this strips any embedded scripts or metadata. For documents: check for embedded macros, scripts, or suspicious content. For any file: verify the content type matches the extension, reject files with executable content, and log any suspicious upload attempts with the user's ID and IP address.
```

COPY THIS PROMPT

Prompt 5: Prevent denial-of-service via uploads

```
Add protections against upload-based denial-of-service: (1) Set a maximum file size at the server/middleware level (not just the frontend). (2) Limit the number of files a user can upload per minute. (3) Limit total storage per user if applicable. (4) Process uploads asynchronously so a large file doesn't block your server. (5) Add timeout limits on upload requests. Show me where each limit is enforced.
```

COPY THIS PROMPT

Tip: Try uploading a .txt file renamed to .jpg. If your app accepts it, your file type validation is based on the extension only — which is trivially easy to fake. Real validation reads the file's actual content.

8 RATE LIMITING & BRUTE FORCE

AI tools build login forms and API endpoints with no concept of "too many attempts." That means an attacker can try thousands of password combinations per second on your login form, create millions of fake accounts, spam your contact form, or call your API endpoints until your server crashes or your bill skyrockets.

Rate limiting means setting a cap on how many times someone can do something in a given time period. It's one of the simplest and most effective defenses, and AI tools almost never add it on their own.

What can go wrong

An attacker writes a script that tries 10,000 common passwords against your login endpoint — one after another, automatically. With no rate limiting, your server happily processes every single attempt. If any user has a weak password, the attacker is in. This is called a brute force attack, and it works because most apps have no limit on failed login attempts. It's even worse if the app confirms which usernames exist via error messages like 'incorrect password' vs 'user not found.'

Prompt 1: Add rate limiting to login

COPY THIS PROMPT

```
Add rate limiting to the login endpoint. After 5 failed attempts from the same IP address, block further login attempts from that IP for 15 minutes. After 10 failed attempts for the same account (regardless of IP), temporarily lock the account and require email verification to unlock. Always return the same error message for wrong username and wrong password — 'Invalid email or password' — so attackers can't tell which one was wrong.
```

Prompt 2: Add rate limiting to API endpoints

COPY THIS PROMPT

```
Add rate limiting to all API endpoints in this app. Set reasonable limits based on expected usage: (1) Authentication endpoints (login, signup, password reset): 5-10 requests per minute per IP. (2) Read endpoints (viewing data): 60-100 requests per minute per user. (3) Write endpoints (creating/updating data): 20-30 requests per minute per user. (4) File upload endpoints: 5-10 per minute per user. Return a 429 Too Many Requests response when limits are exceeded, with a Retry-After header.
```

Prompt 3: Protect signup and password reset from abuse

COPY THIS PROMPT

```
Add abuse protection to the signup and password reset endpoints. For signup: (1) Rate limit to 3 accounts per IP per hour. (2) Add email verification before the account is fully active. (3) Add a honeypot field to catch bots. For password reset: (1) Rate limit to 3 reset requests per email per hour. (2) Don't reveal whether the email exists in the system. (3) Rate limit the actual reset endpoint to prevent token brute-forcing.
```

Prompt 4: Add account lockout with notification

COPY THIS PROMPT

```
After too many failed login attempts for a specific account, temporarily lock the account. Send an email to the account owner saying 'Someone tried to log into your account multiple times. If this wasn't you, please change your password.' Include: the time of the attempts, the approximate location (based on IP), and a direct link to change their password. Unlock the account automatically after 30 minutes or when the user clicks the email link.
```

Prompt 5: Protect expensive operations

COPY THIS PROMPT

```
Identify every endpoint that triggers an expensive operation — sending emails, making API calls to external services, generating AI responses, processing images, running database-heavy queries. Add specific rate limits to each one. For example: email-sending endpoints should be limited to 10 per hour per user. AI/LLM endpoints should be limited based on your token budget. Any endpoint that costs you money per call needs its own limit.
```

Tip: Use a library like express-rate-limit (Node.js), slowapi (Python/FastAPI), or your hosting platform's built-in rate limiting. Don't build rate limiting from scratch — use a battle-tested library.

9 HTTPS & TRANSPORT SECURITY

HTTPS encrypts the data traveling between your user's browser and your server. Without it, everything — passwords, personal data, session tokens — is sent in plain text that anyone on the same network can read. Most hosting platforms offer free HTTPS through Let's Encrypt, but AI tools often generate code that doesn't enforce it properly: mixed content warnings, HTTP redirects that don't happen, API calls over plain HTTP.

> **What can go wrong**
>
> A user logs into your app from a coffee shop WiFi. Your app uses HTTP for its API calls even though the main page loads over HTTPS. An attacker on the same network runs a simple tool that intercepts all HTTP traffic — they capture the user's session token, personal data, and every API request. They can now impersonate that user. This is called a man-in-the-middle attack, and it's trivially easy on unsecured networks.

Prompt 1: Check for HTTPS enforcement

COPY THIS PROMPT

```
Review this entire app for HTTPS issues. Check: (1) Does the server redirect all HTTP requests to
HTTPS? (2) Are all API calls using https:// URLs, not http://? (3) Are all external resources
(images, scripts, stylesheets, fonts) loaded over HTTPS? (4) Is the Strict-Transport-Security
(HSTS) header set to force browsers to always use HTTPS? (5) Are WebSocket connections using
wss:// not ws://? List every issue and fix it.
```

Prompt 2: Fix mixed content issues

COPY THIS PROMPT

```
Scan all frontend code, CSS, and HTML for any resources loaded over plain HTTP (http://). These
cause 'mixed content' warnings and can be blocked by browsers. Change every http:// URL to
https:// or use protocol-relative URLs (//). Check: images, scripts, stylesheets, fonts, iframes,
API endpoints, and WebSocket connections. Also check any hardcoded URLs in the database or
environment variables.
```

Prompt 3: Add security headers

COPY THIS PROMPT

```
Add these HTTP security headers to every response from this app: (1) Strict-Transport-Security:
max-age=31536000; includeSubDomains (enforce HTTPS for a year). (2) Content-Security-Policy:
restrict where scripts, styles, and resources can load from. (3) X-Content-Type-Options: nosniff
(prevent MIME type sniffing). (4) X-Frame-Options: DENY (prevent clickjacking). (5)
Referrer-Policy: strict-origin-when-cross-origin. (6) Permissions-Policy to disable unused browser
features (camera, microphone, geolocation). Show me exactly how to add these for [your framework].
```

Prompt 4: Secure cookie transport

COPY THIS PROMPT

```
Review all cookies set by this app. Make sure every cookie has: (1) The 'secure' flag so it's only
sent over HTTPS. (2) The 'httpOnly' flag so JavaScript can't access it (for session cookies). (3)
The 'sameSite' attribute set to 'strict' or 'lax.' (4) A reasonable expiration time. Also check if
any sensitive data is being stored in cookies that should be in server-side sessions instead. List
every cookie and its current flags.
```

Tip: Most hosting platforms (Vercel, Netlify, Railway) handle HTTPS certificates automatically. But you still need to make sure your code enforces HTTPS everywhere — the platform handles the certificate, your code handles the redirects and headers.

10 DATA PRIVACY & PII HANDLING

If your app collects personal information — names, emails, addresses, phone numbers, payment info, health data — you have a legal and ethical responsibility to protect it. AI tools will store personal data wherever is most convenient, usually in plain text with no encryption, no access controls, and no plan for deletion. Privacy laws like GDPR (Europe), CCPA (California), and others require that you handle personal data carefully.

What can go wrong

Your app collects email addresses and stores them in a database. Your AI tool also logs every API request to the console, including request bodies with user emails and passwords. A data breach exposes these logs. Now you've leaked personal data that you were storing unnecessarily, and you may be legally required to notify every affected user and the relevant authorities. Under GDPR, fines can reach 4% of annual revenue.

Prompt 1: Map all personal data in this app

```
Create a complete inventory of every place this app collects, stores, or transmits personal data
(PII). For each piece of data, tell me: (1) What is it (name, email, address, etc.)? (2) Where is
it collected (which form/endpoint)? (3) Where is it stored (which database table/field)? (4) Who
can access it? (5) Is it encrypted at rest? (6) Is it transmitted securely? (7) Is there a way to
delete it if the user requests? List everything — I need the complete picture.
```

Prompt 2: Minimize data collection

```
Review every form and data collection point in this app. For each field, ask: do I actually need
this data to provide the service? Remove any fields that aren't strictly necessary. For fields
that are necessary, check if I can store a less sensitive version (e.g., last four digits of phone
number instead of the full number, hashed email for analytics instead of plain text). Apply the
principle of data minimization — collect only what you need.
```

Prompt 3: Encrypt sensitive data at rest

```
Review how sensitive data is stored in the database. Check: (1) Are passwords hashed with bcrypt
or argon2 (not MD5 or SHA-1)? (2) Are other sensitive fields (SSN, payment info, health data)
encrypted at rest using AES-256 or equivalent? (3) Where are the encryption keys stored — are they
separate from the database? (4) Is the database itself encrypted at the storage level? If not,
enable it. List everything that needs encryption and add it.
```

Prompt 4: Add a user data deletion endpoint

```
Build a feature that lets users delete their account and all associated data. This should: (1)
Delete all records in every table that reference this user (cascade through all relationships).
(2) Remove their files from storage. (3) Remove their data from any logs or analytics. (4) Send a
confirmation email. (5) Actually delete the data — not just mark it as inactive. (6) Return
confirmation that all data was removed. This is required by GDPR and similar laws.
```

Prompt 5: Add a privacy-aware data export

COPY THIS PROMPT

```
Build a feature that lets users download all the data your app holds about them (GDPR calls this
'data portability'). Generate a JSON or CSV file containing: all their profile information, all
content they created, all activity logs related to them, and all data stored in connected
services. Make sure the export does not include other users' data, internal system fields, or data
about other users that appears in shared contexts.
```

Tip: Even if you're not legally required to comply with GDPR or CCPA, following their principles is good practice. Collect only what you need, encrypt what you store, let users see and delete their data, and have a plan for breach notification.

11 INSECURE CONFIGURATION & DEFAULTS

Every tool, framework, and service your app uses comes with default settings. Many of those defaults are designed for convenience during development — not for security in production. Debug mode, open CORS policies, default database passwords, publicly accessible admin panels. AI tools set things up to work. They don't set things up to be safe.

Prompt 1: Production readiness check

COPY THIS PROMPT

```
Is this project configured for production or still in development mode? Check for: (1) Debug mode
— is it turned off? (2) CORS settings — is the API accepting requests from everywhere ('*') or
only from my app's domain? (3) Database — is it publicly accessible or restricted to the app
server? (4) Default credentials — are there any default passwords, admin accounts, or test
credentials still in place? (5) Error verbosity — are detailed errors being sent to the client?
List everything that needs to change before this goes live.
```

Prompt 2: Lock down CORS

COPY THIS PROMPT

```
Review the CORS configuration for this API. Which domains are currently allowed to make requests?
If it's set to '*' (allow all origins), change it to only allow requests from my app's domain:
[your-domain.com]. Also check that only the necessary HTTP methods (GET, POST, PUT, DELETE) are
allowed — not every method by default. And verify that credentials are not being allowed with
wildcard origins.
```

Prompt 3: Database security review

COPY THIS PROMPT

```
Review the database configuration for production readiness. Check: (1) Is the database publicly
accessible from the internet, or is it restricted to connections from the app server only? (2) Are
there any default or weak credentials? (3) Is the connection using SSL/TLS? (4) Is row-level
security enabled if the platform supports it? (5) Are there any overly permissive access policies?
(6) Are database backups encrypted? Fix anything that's too open.
```

Prompt 4: Remove development artifacts

COPY THIS PROMPT

```
Scan this entire project for anything that should not be in production: (1) Test accounts or dummy
data in the database. (2) Debug flags, verbose logging, or development-only code paths. (3) API
documentation endpoints (like /api/docs or /swagger) that should be disabled or password-protected
in production. (4) Any TODO comments mentioning security. (5) Test API keys or placeholder
credentials. (6) Development-only middleware or routes. Remove or disable everything you find.
```

Prompt 5: Review third-party service configuration

```
Review every third-party service connected to this app (database, auth provider, email service,
payment processor, file storage, analytics). For each one, check: (1) Are you using the production
configuration, not development/sandbox? (2) Are API keys restricted to only the permissions they
need (principle of least privilege)? (3) Are webhook endpoints verified with a signature? (4) Are
any services using default or overly permissive settings? List each service and what needs to
change.
```

COPY THIS PROMPT

Tip: Create a pre-deployment checklist and run through it before every launch. Many production incidents come from forgetting to change one setting — a checklist prevents that.

12 OUTDATED & VULNERABLE DEPENDENCIES

Your app depends on dozens or hundreds of open-source packages — libraries other people built. Those packages sometimes have security vulnerabilities discovered after you installed them. If you never update, you're running code with known holes that are publicly documented and easy to exploit.

Prompt 1: Run a security audit

```
Run a security audit on this project's dependencies. Use the equivalent of 'npm audit' for
JavaScript or 'pip audit' for Python. List every vulnerability found, its severity (low / medium /
high / critical), which package it's in, and what version fixes it. Then apply all safe fixes that
don't require major version changes. For any that do require major version changes, tell me what
might break so I can decide.
```

COPY THIS PROMPT

Prompt 2: Check for abandoned packages

```
Review every dependency in this project. Flag any package where: (1) the last update was more than
12 months ago, (2) the GitHub repository is archived or has no recent activity, or (3) it has
known vulnerabilities with no fix available. For each flagged package, suggest a well-maintained
alternative I should switch to.
```

COPY THIS PROMPT

Prompt 3: Safe update strategy

```
I want to update the dependencies in this project safely. Create a plan that updates one package
at a time, starting with the ones that have known security vulnerabilities. After each update,
tell me what to test to make sure nothing broke. If something breaks, tell me how to roll back
that specific update. Do not update everything at once.
```

COPY THIS PROMPT

Prompt 4: Lock down dependency versions

```
Review the package.json (or equivalent) for this project. Check: (1) Are dependency versions
pinned (exact versions like 2.1.3) or using ranges (^2.1.3 or ~2.1.3)? (2) Is there a lock file
(package-lock.json, yarn.lock, poetry.lock) and is it committed to Git? (3) Are there any
dependencies installed from Git URLs, tarballs, or other non-registry sources that could be
tampered with? Pin all versions and commit the lock file.
```

COPY THIS PROMPT

Prompt 5: Audit transitive dependencies

```
List all transitive (indirect) dependencies in this project — the packages that my direct
dependencies depend on. Are any of them flagged for security vulnerabilities? Are any of them
unmaintained? A vulnerability in a transitive dependency is just as dangerous as one in a direct
dependency. Show me the full dependency tree for any package that has a vulnerable sub-dependency.
```

COPY THIS PROMPT

Tip: Enable GitHub Dependabot on your repository. It automatically checks for vulnerable dependencies and creates pull requests to update them. It's free and takes about 30 seconds to turn on.

13 LOGGING, MONITORING & AUDIT TRAILS

AI tools build features but never add monitoring. That means if someone is trying to break into accounts, exploiting a vulnerability, or accessing data they shouldn't — you have no way of knowing. Good logging tells you what happened, when, and who did it. Good monitoring alerts you when something suspicious is happening right now. Without both, breaches go undetected for weeks or months.

> **What can go wrong**
>
> An attacker finds a vulnerability and accesses 10,000 user records over three weeks. They're careful — they space out requests to avoid suspicion. Because your app has no logging of data access and no monitoring for unusual patterns, you don't discover the breach until a user reports their data appearing on a dark web marketplace. By then, the damage is done and you have no logs to investigate what happened or how.

Prompt 1: Add security event logging

```
Add logging for these security-relevant events: (1) All login attempts (successful and failed)
with timestamp, IP, and user agent. (2) All failed authorization attempts (someone trying to
access a resource they don't own). (3) All account changes (password change, email change, role
change). (4) All admin actions. (5) All data exports or bulk data access. (6) All API errors above
a certain rate. Store these logs in a secure location that can't be modified by an attacker who
compromises the app.
```

COPY THIS PROMPT

Prompt 2: Set up alerts for suspicious activity

```
Add monitoring alerts for: (1) More than 10 failed login attempts for a single account in 5
minutes. (2) A single IP making requests to more than 50 different user accounts. (3) Bulk data
access (more than 100 records in a single query or rapid sequential requests). (4) Login from a
new country or unusual IP for an existing user. (5) Multiple password reset requests in a short
period. (6) Any 403/401 errors above baseline. Tell me how to set up email or Slack notifications
for each alert.
```

COPY THIS PROMPT

Prompt 3: Create an audit trail for sensitive data

```
For every piece of sensitive data in this app (user profiles, payment info, admin settings, user
content), add an audit trail that records: (1) Who accessed or modified the data. (2) When they
accessed or modified it. (3) What the previous value was (for modifications). (4) The IP address
and session ID. (5) Whether the access was through the UI or directly via API. Store audit logs in
a separate, append-only table that can't be modified or deleted.
```

COPY THIS PROMPT

Prompt 4: Add health monitoring and error tracking

```
Set up application health monitoring: (1) Track error rates — alert if errors spike above normal
levels. (2) Monitor response times — alert if endpoints suddenly get slower (could indicate a DDoS
or resource exhaustion attack). (3) Track uptime and set up a status page. (4) Monitor disk space
and database connection count. (5) Set up structured logging with severity levels (info, warn,
error, critical). Tell me which free or low-cost monitoring tools to use for [my hosting
platform].
```

COPY THIS PROMPT

Prompt 5: Ensure logs don't leak sensitive data

```
Review all logging in this application. Check that logs never contain: (1) Passwords or password
hashes. (2) Full API keys or tokens. (3) Credit card numbers or full SSNs. (4) Session tokens or
authentication cookies. (5) Full request bodies that include personal data. Replace sensitive
values in logs with redacted versions (e.g., 'sk_live_****1234'). Verify that log files are not
publicly accessible and have appropriate access controls.
```

COPY THIS PROMPT

Tip: Free tools like Sentry (error tracking), UptimeRobot (uptime monitoring), and your hosting platform's built-in logs are enough to start. You don't need expensive enterprise monitoring — you just need to be logging the right events.

14 THE MASTER SECURITY REVIEW

This is the one prompt that matters most. Run it after every meaningful feature you build. Don't try to memorize everything in this pack — just build your feature, then paste this prompt. The AI will catch what it missed the first time. Run it twice. It finds layers — the second pass always catches things the first pass didn't.

THE PROMPT: Run this after every feature

```
I just finished building [describe the feature]. Review only the new code for security issues.
Check each of these specifically: (1) Are there permission checks on every endpoint — both
authentication (is the user logged in?) and authorization (are they allowed to access this
specific resource)? (2) Are there any hardcoded secrets, API keys, or tokens? (3) Is user input
validated on the backend, not just the frontend? (4) Are error messages safe — no stack traces,
file paths, or database details exposed to the user? (5) Is all user-generated content sanitized
before being stored or displayed? (6) Are database queries using parameterized queries, not string
concatenation? (7) If there are file uploads, are they validated and stored securely? (8) Is there
rate limiting on any endpoint that could be abused? (9) Are there any CSRF vulnerabilities in
state-changing operations? (10) Is sensitive data encrypted and not over-exposed in API responses?
Flag everything you find, fix it, and then tell me what you changed and why.
```

COPY THIS PROMPT

Why run it twice?

When the AI fixes a security issue, it sometimes introduces a new pattern that has its own vulnerability. Or fixing one issue reveals another that was hidden behind it. The second pass catches these cascading issues. Professional security audits work the same way — nobody finds everything on the first pass. Run it until the AI comes back with nothing to fix. That's when you're done.

THE PRE-LAUNCH PROMPT: Run this before you go live

COPY THIS PROMPT

```
I'm about to deploy this app to production. Do a comprehensive security review of the entire codebase. Check: (1) All secrets are in environment variables, not in code. (2) All user input is validated on the backend. (3) All database queries use parameterized queries. (4) All API endpoints have authentication and authorization checks. (5) Error messages don't expose internal details. (6) CORS is locked to my domain only. (7) Debug mode is off. (8) All cookies have secure, httpOnly, and sameSite flags. (9) HTTPS is enforced everywhere. (10) Rate limiting is in place on login and sensitive endpoints. (11) File uploads are validated and stored securely. (12) Dependencies have no known critical vulnerabilities. (13) No test credentials, dummy data, or development artifacts remain. Give me a pass/fail for each item and fix anything that fails.
```

The Security Review Workflow

1. Build the feature. Get it working first — don't worry about security while iterating.
2. Paste the master security review prompt above.
3. Let the AI find and fix issues. It usually catches 2-3 on the first pass.
4. Run the same prompt again. It will find a second layer.
5. Repeat until the AI comes back clean. Then move on to the next feature.

The 6 questions to ask yourself with every feature

1. Who is allowed to use this — and does my code actually check?
2. What happens if someone types something weird into this field?
3. What sensitive data am I touching — and is it stored, sent, and shown safely?
4. Am I using secure defaults, or did I just keep whatever the AI gave me?
5. If someone tried to abuse this feature, what would they do first?
6. Would I know if something went wrong — is there logging and monitoring?

Want to go deeper?

This prompt pack covers the most common security gaps in AI-generated code. If you want to understand the full picture — why these vulnerabilities exist, how to think about security as a mindset (not just a checklist), and how to plan, build, and ship a real app from scratch — check out Zero to Dangerous: Vibe Coding Fundamentals at teknical.ai. It's the course that teaches you the foundations AI tools assume you already know.